ch-3 Personal Project — Ideas & Toolkit

kokoye2007

2026-06-18

ch-3 Personal Project — Ideas & Toolkit

> Pick **ONE** idea, keep it small, and make sure it uses **1 MCP + 1 skill + 1 agent**.
> Requirements recap: public repo · `.mcp.json` · `.claude/skills/<name>/SKILL.md` · `.claude/agents/<name>.md` · 6×20 Marp slides · `report.md` in your team repo.
>
> Companion docs: apis.md (free APIs) · GITHUB_SEARCH.md (find real examples) · ch3-personal-project-guide.md (step-by-step).

30-minute quickstart

1. **Pick one idea** from the categories below. Smaller = better.
2. **Make a public repo** on your GitHub, clone it, `cd` in.
3. **Add** `.mcp.json` (copy a snippet below). Run `claude` — confirm the MCP tools load.
4. **Add one skill** at `.claude/skills/<name>/SKILL.md` and **one agent** at `.claude/agents/<name>.md`.
5. **Build the smallest working version.** Commit often (small, frequent commits = your methodology).
6. **Write** `report.md` + **6×20 Marp slides** as you go, not at the end.

> Tip: search GitHub for how others did it first — see GITHUB_SEARCH.md. `gh search code "filename:.mcp.json"` shows real configs you can copy.

How to add the three (toolkit basics)

MCP — gives Claude tools (files, web, GitHub, browser, DB...). Add a `.mcp.json` at repo root:

```
{
  "mcpServers": {
    "filesystem": { "command": "npx", "args": ["-y", "@modelcontextprotocol/
server-filesystem", "."] }
  }
}
```

Docs: https://modelcontextprotocol.io · servers list: https://github.com/modelcontextprotocol/servers

Skill — a reusable instruction pack Claude loads on demand. Create `.claude/skills/<name>/SKILL.md`:

```
---
name: release-notes
description: Use when the user wants a changelog from recent git commits.

---

# Release notes

1. Run `git log --oneline <last-tag>..HEAD`.
2. Group commits by type (feat / fix / docs).
3. Write a short markdown changelog with a date heading.
```

Use it for a repeatable task in your project (e.g. “generate a release note”, “lint my data”).

Agent — a sub-assistant with its own tools/prompt for one job. Create `.claude/agents/<name>.md`:

```
---
name: data-cleaner
description: Normalizes and de-duplicates CSV/JSON rows before reporting.

tools: Read, Write, Bash
---

You clean datasets. Trim whitespace, fix types, drop duplicate rows,
and report how many rows changed. Never invent data.
```

Use it for a focused role (e.g. `data-cleaner`, `test-writer`, `pr-reviewer`).

Reference: https://docs.claude.com/en/docs/claude-code (skills, agents, MCP, settings)

Project ideas by category

Each category lists: **idea examples**, a fitting **MCP**, a **skill** idea, an **agent** idea, and **resources**.

1. Web app / site

- **Ideas:** personal portfolio, event RSVP page, small dashboard, link-in-bio, study-tracker, habit tracker.
- **MCP:** `filesystem`, `fetch` (pull live data), `chrome-devtools` / `playwright` (verify UI).
- **Skill:** `deploy-checklist` (build → test → publish steps).
- **Agent:** `ui-reviewer` (checks accessibility + responsive).
- **Resources:** Vite https://vitejs.dev · Tailwind https://tailwindcss.com · GitHub Pages https://pages.github.com

2. CLI tool / automation

- **Ideas:** file renamer, markdown→PDF, repo stats, daily-standup generator, backup script, image optimizer.
- **MCP:** `filesystem`, `git` / `github`.
- **Skill:** `release-notes` (summarize commits since last tag).
- **Agent:** `refactor-cleaner` (dead-code/format pass).
- **Resources:** Node https://nodejs.org · Bun https://bun.sh · commander https://github.com/tj/commander.js

3. Data / scraper / report

- **Ideas:** price tracker, news digest, CSV cleaner + chart, survey summarizer, weather logger.
- **MCP:** `fetch`, `filesystem`, a DB MCP (sqlite/postgres).
- **Skill:** `data-validate` (schema + sanity checks).
- **Agent:** `data-cleaner` (normalize + dedupe rows).
- **Resources:** free APIs → apis.md · Playwright https://playwright.dev · DuckDB https://duckdb.org · Pandas https://pandas.pydata.org

4. Bot / chat / integration

- **Ideas:** Telegram reminder bot, Discord FAQ bot, webhook notifier, LINE helper, RSS-to-chat.

- **MCP:** `fetch`, `filesystem`; provider SDK as needed.
- **Skill:** `reply-templates` (consistent message formats).
- **Agent:** `triage` (classify incoming messages).
- **Resources:** grammY (TG) https://grammy.dev · discord.js https://discord.js.org

5. API / backend service

- **Ideas:** URL shortener, notes API, image-resize service, auth demo, QR generator.
- **MCP:** a DB MCP, `fetch`.
- **Skill:** `api-contract` (keep routes + docs in sync).
- **Agent:** `endpoint-tester` (hits routes, checks status/shape).
- **Resources:** Hono https://hono.dev · Express https://expressjs.com · FastAPI https://fastapi.tiangolo.com

6. Content / docs / media

- **Ideas:** blog generator, slide-maker, study-notes site, changelog automation, flashcard maker.
- **MCP:** `filesystem`, `fetch`.
- **Skill:** `style-guide` (tone + formatting rules).
- **Agent:** `doc-updater` (regenerate docs from code).
- **Resources:** Marp https://marp.app · Astro https://astro.build · Pandoc https://pandoc.org

---

Scoping tips

- **Smaller is better.** One clear feature done well > a half-built platform.
- **Vertical slice.** Make one full path work end-to-end before adding a second feature.
- **Commit as you build** (small, frequent commits — this is your methodology).
- The **agent + skill + MCP must actually be used** in your work, not just present as empty files — describe where in `report.md`.

Common mistakes (avoid these)

- ❌ Empty/placeholder skill or agent file that's never used. The checker sees the file; reviewers see it does nothing.
- ❌ Hard-coded API keys in the repo. Use `.env` + `.gitignore`. (See apis.md.)
- ❌ Slides that aren't 6×20 Marp (`auto-advance: 20`). Wrong count/format fails.

- ❌ `report.md` in the wrong place. It goes in your **team repo** at `ch-3/<your-github-username>/report.md`.
- ❌ One giant commit at the end. Show your process with many small commits.
- ❌ Scope too big — you run out of time and nothing works end-to-end.

Done checklist (mirror of the validator)

- [ ] public repo, owner = your GitHub username
- [ ] `.mcp.json` present and used
- [ ] a `.claude/skills/<name>/SKILL.md` you actually used
- [ ] a `.claude/agents/<name>.md` you actually used
- [ ] methodology written in `report.md`
- [ ] 6×20 Marp slides committed (`auto-advance: 20`)
- [ ] `report.md` added to team repo at `ch-3/<your-github-username>/report.md`